

THE REAL-WORLD STORY

Searching a contact in a phonebook with 1,000,000 entries page by page requires 1,000,000 checks in the worst case!

THE HASHMAP AHA MOMENT

What if every contact name instantly transformed into an exact page index via a mathematical hash function? Lookup time drops from **O(N)** to **O(1)**!

WHY DO ARRAYS ALONE FAIL?

- **Array Index Access:** Fast $O(1)$ if index `arr[i]` is known.
- **Array Search by Value:** Slow $O(N)$ linear scan required.
- **Array Insertion at Start:** Slow $O(N)$ shift of all elements!
- **HashMap Solution:** Sacrifices memory to map `Value → Index` for instantaneous $O(1)$ lookups!

HASH FUNCTION MECHANICS

Key → Hash Code → Bucket Index:

Key: "alice"
↓ Hash Function: `hash("alice") = 153029`
Bucket Index = `153029 % array_size (16) = 5`
↓ Direct Array Access!
Bucket[5] stores ("alice", phoneNumber)
O(1) Instant Lookup!

PREREQUISITES & COMPLEXITY REASONING

Operation / Structure	Time Complexity	Auxiliary Space
Array Index Access (<code>arr[i]</code>)	O(1)	O(N) contiguous memory
Linear Search / Scan	O(N)	O(1) space
HashMap Lookup / Put	O(1) average	O(N) space
Prefix Sum Array Query	O(1) per range query	O(N) prefix array

CLUES THAT SCREAM HASHING

1. "Find a pair adding to Target" → HashMap storing `(Target - num) → Index`
2. "Check for duplicates or count frequencies" → HashSet or Frequency Array `int[26]`
3. "Group anagrams together" → HashMap storing `SortedString → List`

PATTERN 1 COMPLEMENT HASHING (Two Sum — LC 1)

WHEN TO USE

Finding two numbers in unsorted array adding to target in single pass $O(N)$ time.

TEMPLATE

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        if (map.containsKey(target - nums[i])) return new int[]{map.get(target - nums[i]), i};
        map.put(nums[i], i);
    }
    return new int[0];
}
```

PATTERN 2 HASHSET DUPLICATE (LC 217)

```
public boolean containsDuplicate(int[] nums) {  
    Set<Integer> set = new HashSet<>();  
    for (int n : nums) if (!set.add(n)) return true;  
    return false;  
}
```

PATTERN 4 GROUP ANAGRAMS (LC 49)

SORTED STRING KEY HASHING

```
public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        char[] ca = s.toCharArray(); Arrays.sort(ca);
        String key = String.valueOf(ca);
        map.computeIfAbsent(key, k → new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}
```

PATTERN 5 TOP K FREQUENT BUCKET SORT (LC 347)**FREQUENCY ARRAY BUCKETS O(N)**

```
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> cnt = new HashMap<>();
    for (int n : nums) cnt.put(n, cnt.getOrDefault(n, 0) + 1);
    List<Integer>[] b = new List[nums.length + 1];
    for (int key : cnt.keySet()) {
        int f = cnt.get(key); if (b[f] == null) b[f] = new ArrayList<>(); b[f].add(key);
    }
    int[] res = new int[k]; int idx = 0;
    for (int i = b.length - 1; i >= 0 && idx < k; i--)
        if (b[i] != null) for (int n : b[i]) res[idx++] = n;
    return res;
}
```

 COMPLETE ARRAYS, STRINGS & HASHING PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
1	Two Sum	Complement Hashing	$O(N)$	$O(N)$	E
217	Contains Duplicate	HashSet Add Check	$O(N)$	$O(N)$	E
242	Valid Anagram	Char Frequency Array	$O(N)$	$O(1)$	E
49	Group Anagrams	Sorted Key Map	$O(N \cdot K \log K)$	$O(N \cdot K)$	M
347	Top K Frequent Elements	Bucket Sort Array	$O(N)$	$O(N)$	M
238	Product of Array Except Self	Prefix & Suffix Accumulation	$O(N)$	$O(1)$	M
128	Longest Consecutive Sequence	HashSet Sequence Start Check	$O(N)$	$O(N)$	M

ARRAYS, STRINGS & HASHING

PAGE 9 OF 10

Curated Problem Ladder — Part 2

PROBLEM LADDER · MEDIUM & HARD

🔍 DRY RUN — Longest Consecutive Sequence ([100, 4, 200, 1, 3, 2])

Step	num	set.contains(num - 1)?	Action	currLen	max
1	100	No (99 missing)	Sequence start! Count 100	1	1
2	4	Yes (3 exists)	Skip (not start of sequence)	-	1
3	200	No (199 missing)	Sequence start! Count 200	1	1
4	1	No (0 missing)	Sequence start! Count 1 → 2 → 3 → 4	4	4 ✓

📐 MATHEMATICAL PROOF — O(N) COMPLEXITY OF LONGEST CONSECUTIVE

Claim: Checking `!set.contains(num - 1)` guarantees total $O(N)$ runtime.

Proof: Inner while loop executes ONLY for the starting element of each sequence. Numbers in the middle of a sequence are skipped in $O(1)$ time. Thus, each array element is visited at most twice $\implies \mathbf{O(N)}$ Total Operations!}\$

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Code Two Sum in 2m using Complement HashMap
- ☐ Check duplicates in $O(N)$ using HashSet `add()`
- ☐ Implement Valid Anagram using `int[26]` array
- ☐ Group Anagrams using sorted char array key
- ☐ Code Top K Frequent in $O(N)$ via Bucket Sort
- ☐ Write Product of Array Except Self without division
- ☐ Code Longest Consecutive Sequence in $O(N)$ time
- ☐ Explain why HashMap lookup is $O(1)$ average

📦 GOLDEN RULES — NEVER FORGET

Rule 1 — Frequency Arrays Outperform HashMaps for Small Alphabets

For lowercase string problems ('a'-'z'), ALWAYS use `int[26]` arrays instead of `HashMap`. Arrays have zero object allocation overhead and 10x faster CPU cache locality!

Rule 2 — Complement Hashing Eliminates Double Loops

Instead of searching for pair (A, B) using nested loops, store $B = \text{Target} - A$ in a HashMap to collapse $O(N^2)$ to $O(N)$!